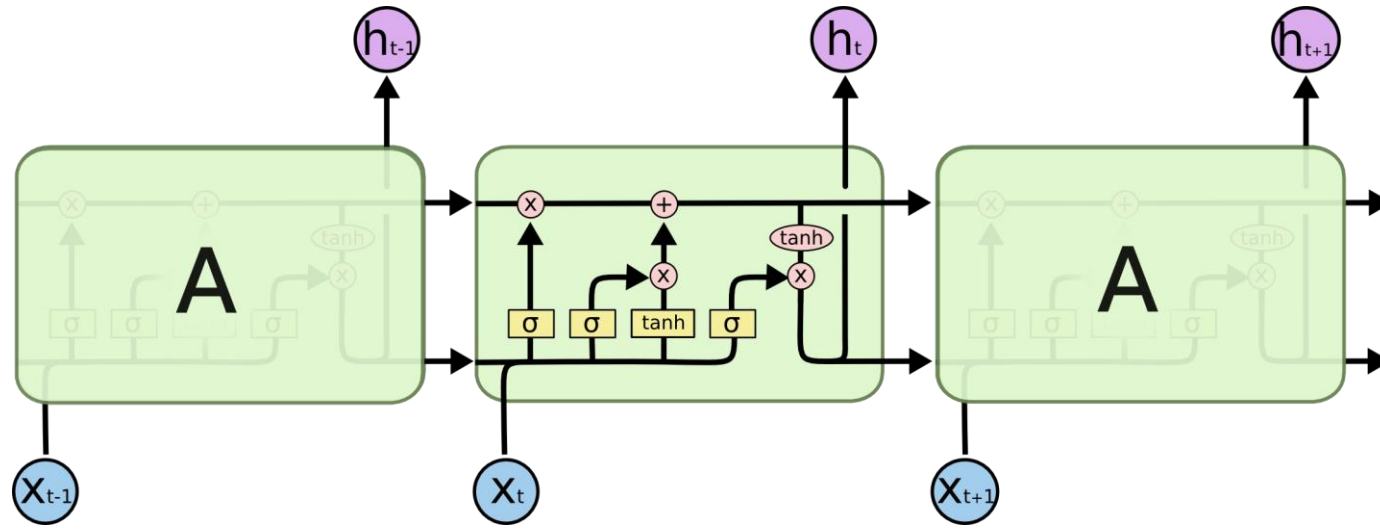
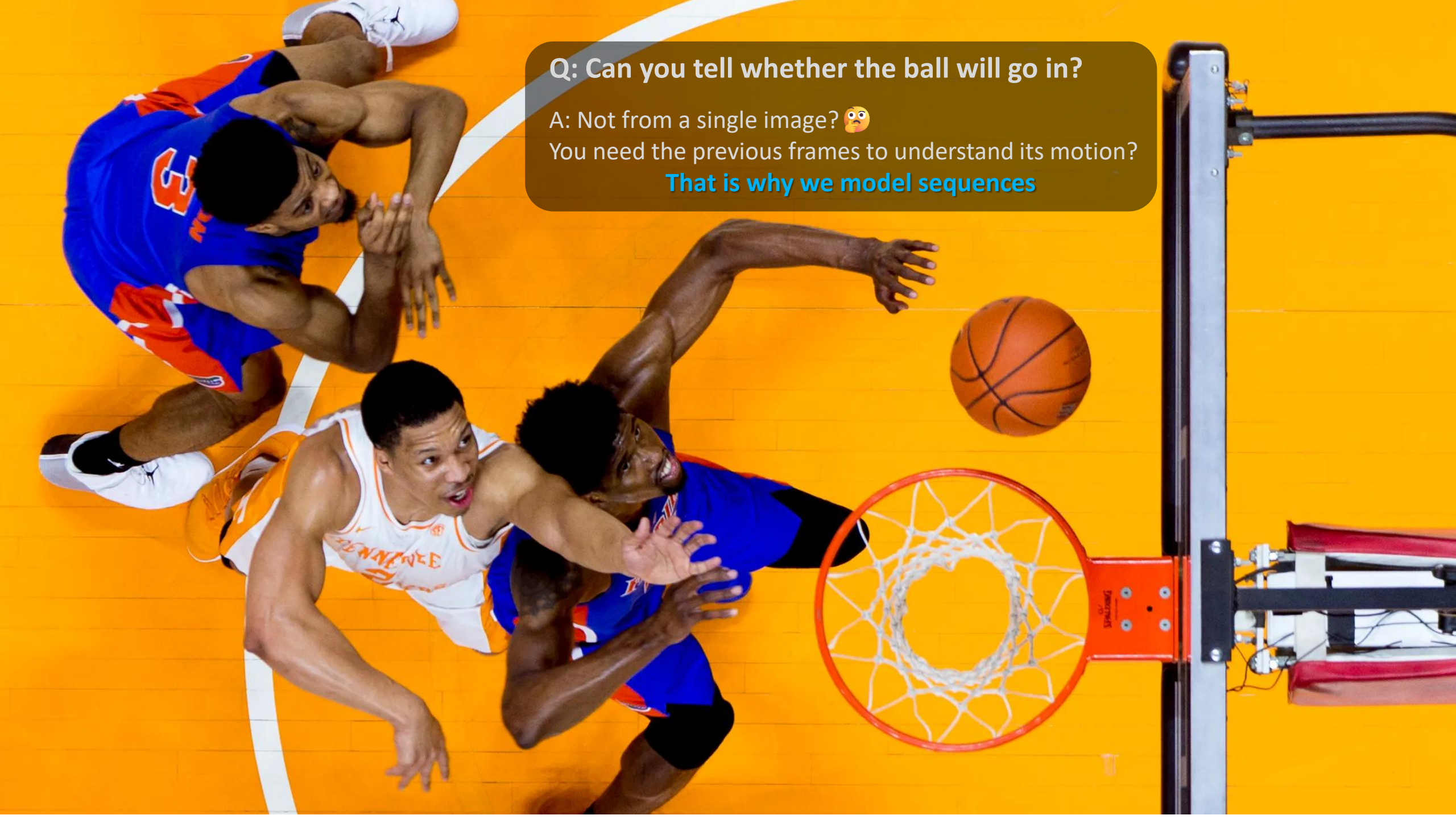




Sequencing Modeling

Introduction and Fundamentals

A high-angle photograph of a basketball game. Three players are visible on a yellow court. One player in a blue jersey with the number 3 is on the left, reaching for a basketball. Another player in a blue jersey is in the center, also reaching for the ball. A third player in a white jersey with orange trim is on the right, looking on. The basketball is in the air, just above the hoop. The hoop is orange with a white net. The background is a solid yellow wall.

Q: Can you tell whether the ball will go in?

A: Not from a single image? 🤔

You need the previous frames to understand its motion?

That is why we model sequences

Sequences in the wild



Speech recognition

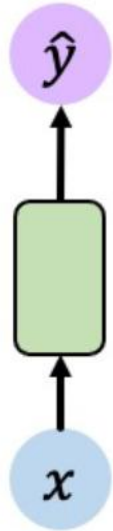


Stock market



Machine Translation

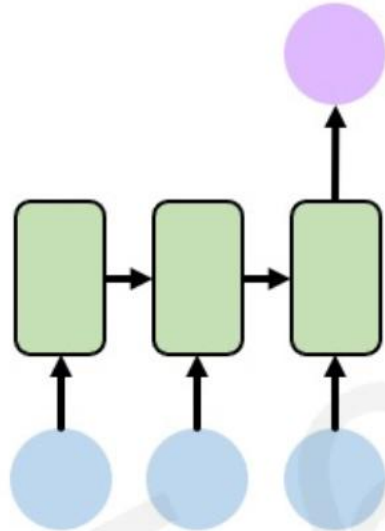
Sequences in the wild



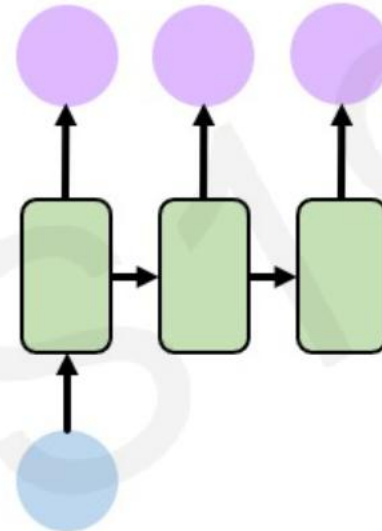
One to One
Binary Classification



"Will I pass this class?"
Student → Pass?



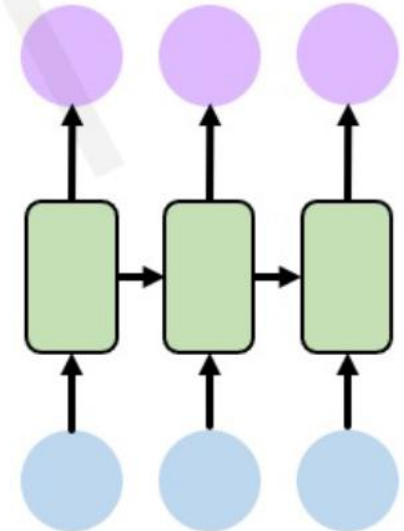
Many to One
Sentiment Classification



One to Many
Image Captioning



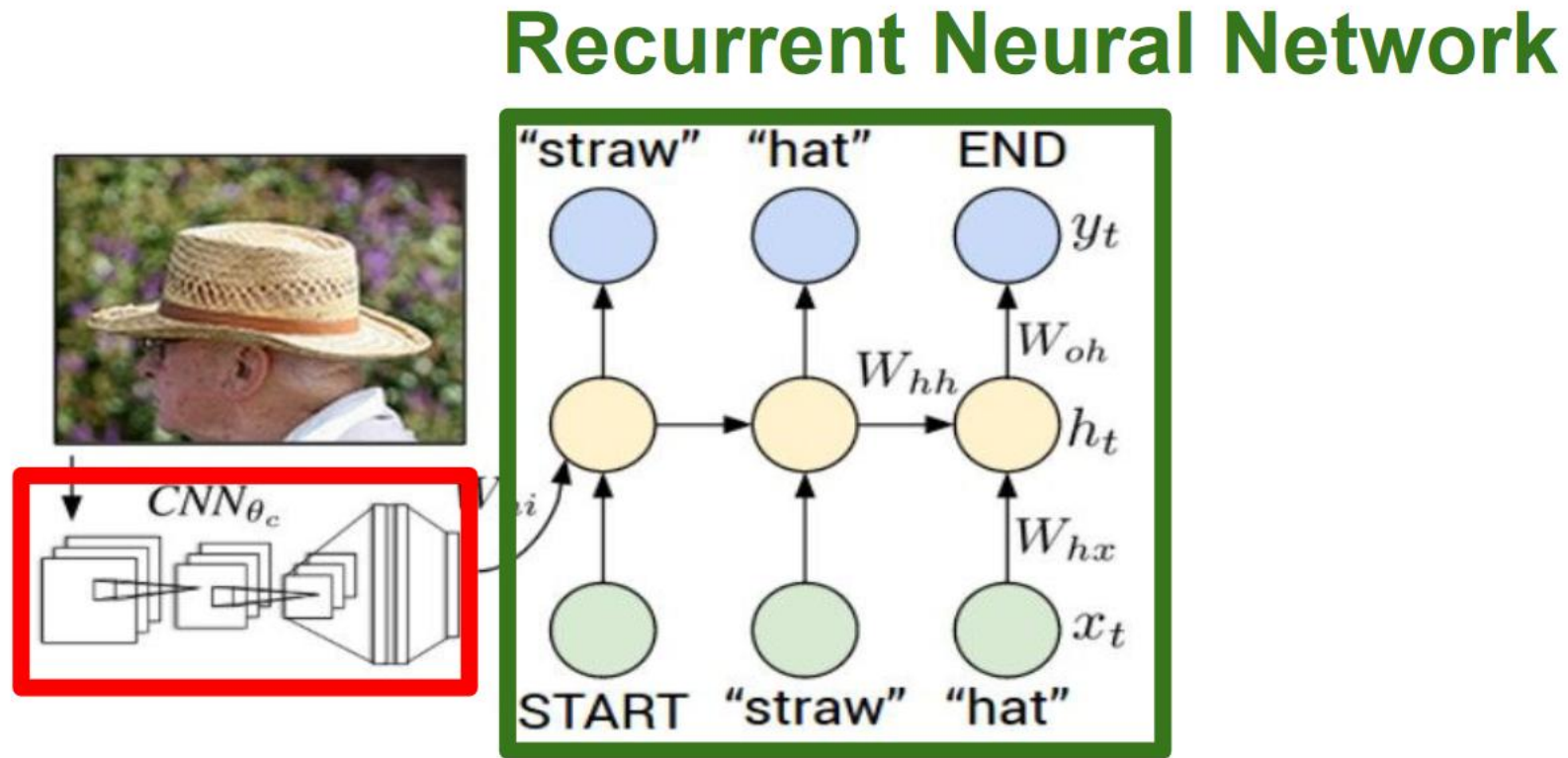
"A baseball player throws a ball."



Many to Many
Machine Translation

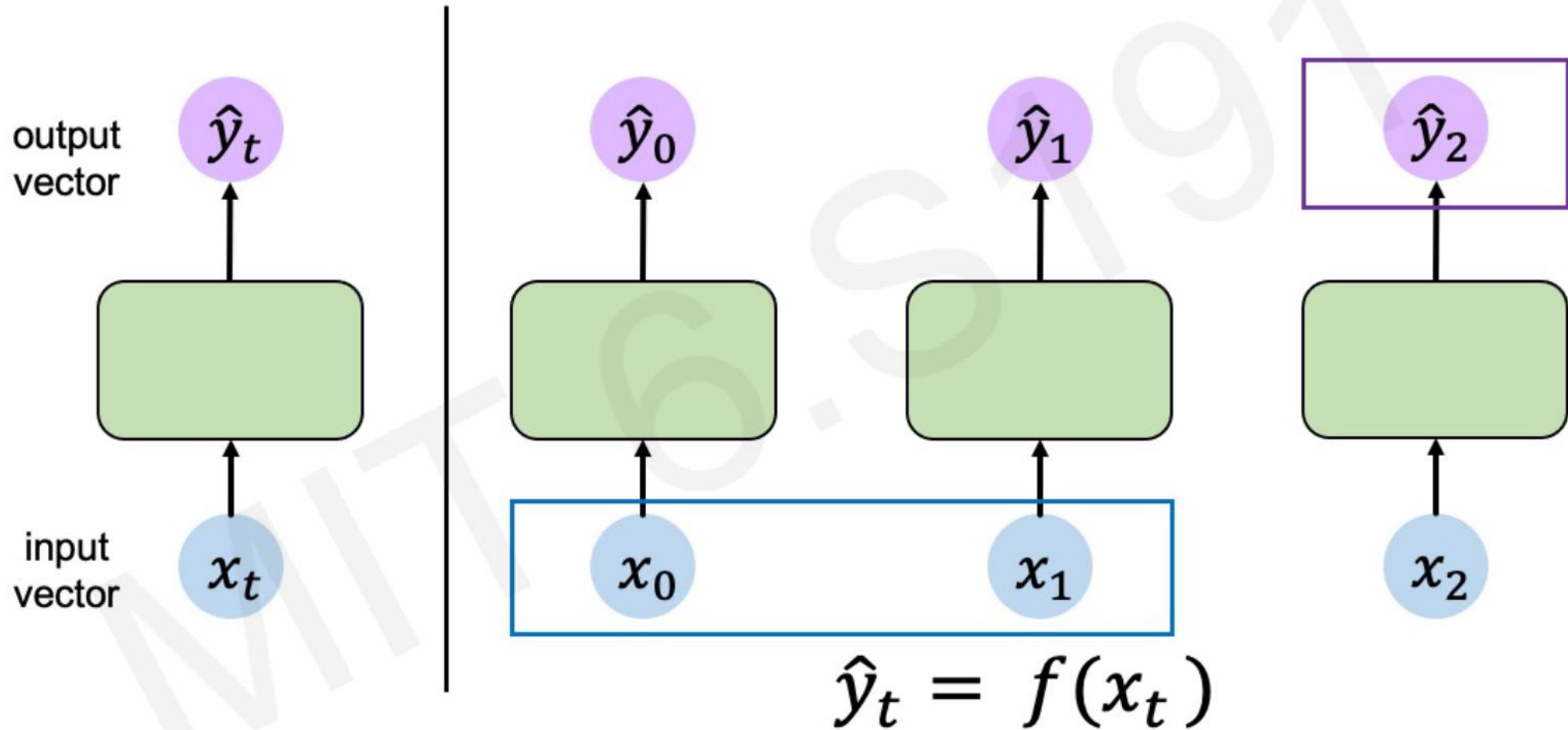


Image Captioning

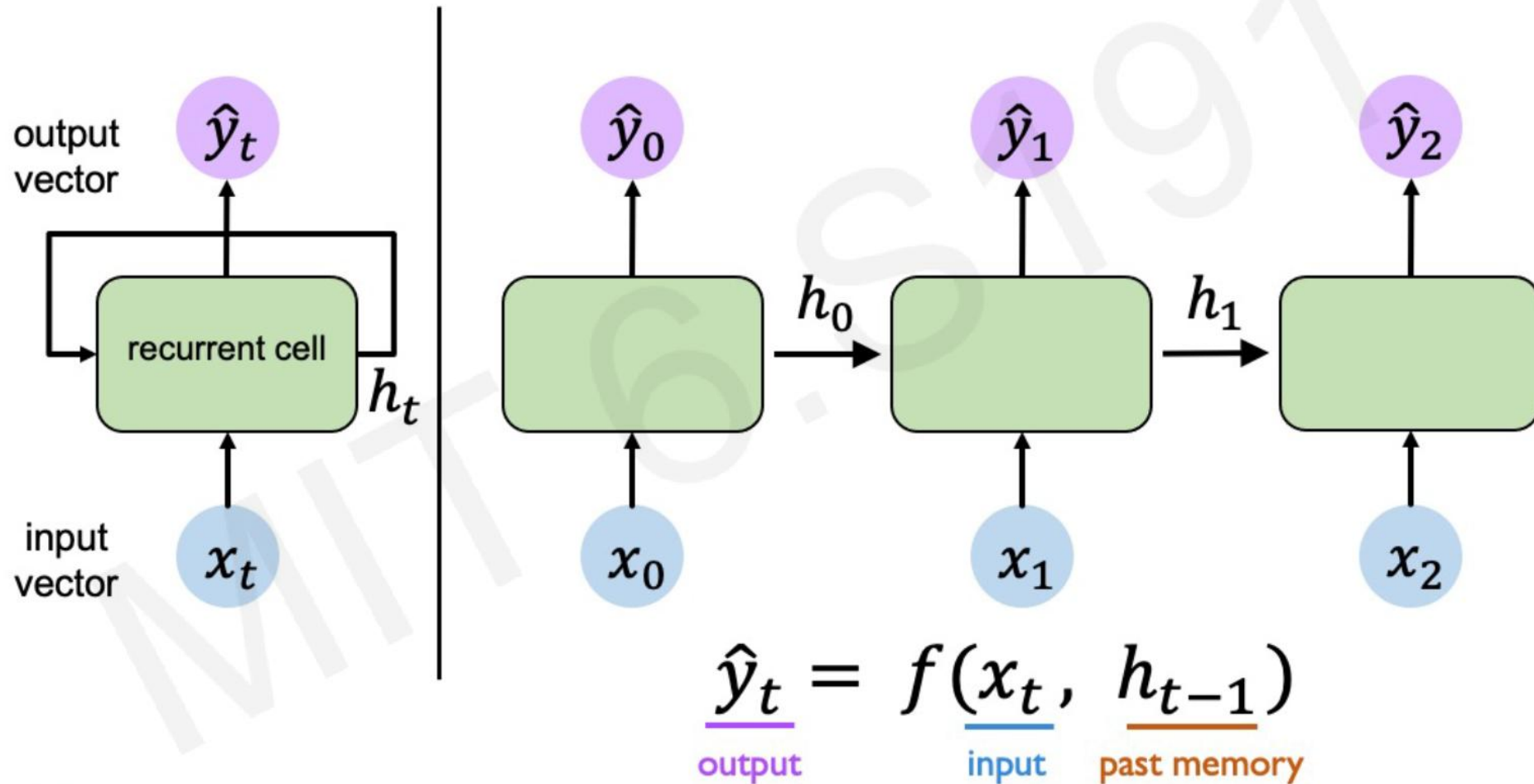


Convolutional Neural Network

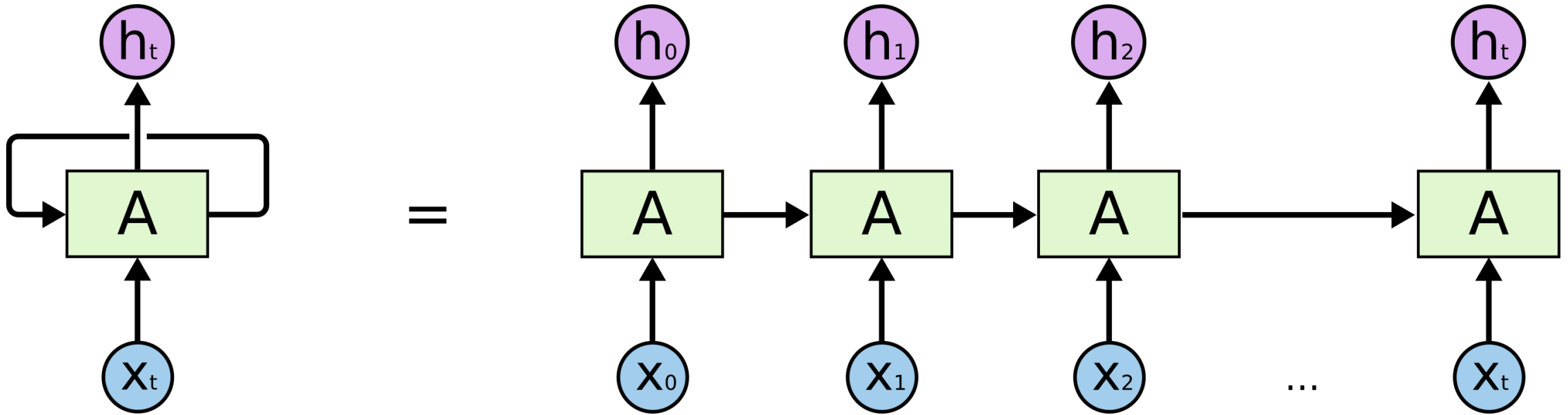
Sequences processing



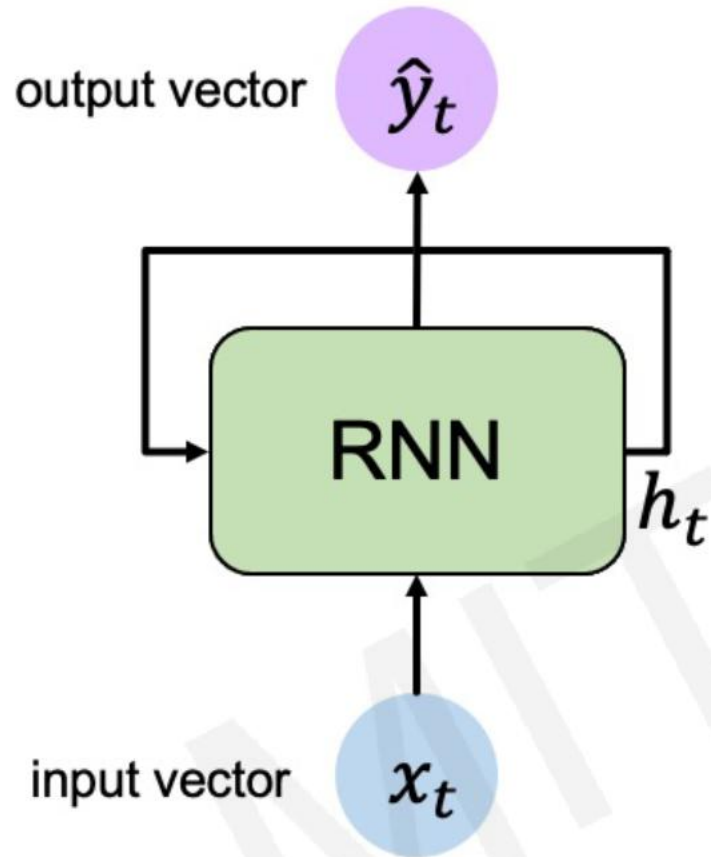
Recurrent Neural Network



An unrolled RNN



Sequences in the wild



Output Vector

$$\hat{y}_t = \mathbf{W}_{hy}^T h_t$$

Update Hidden State

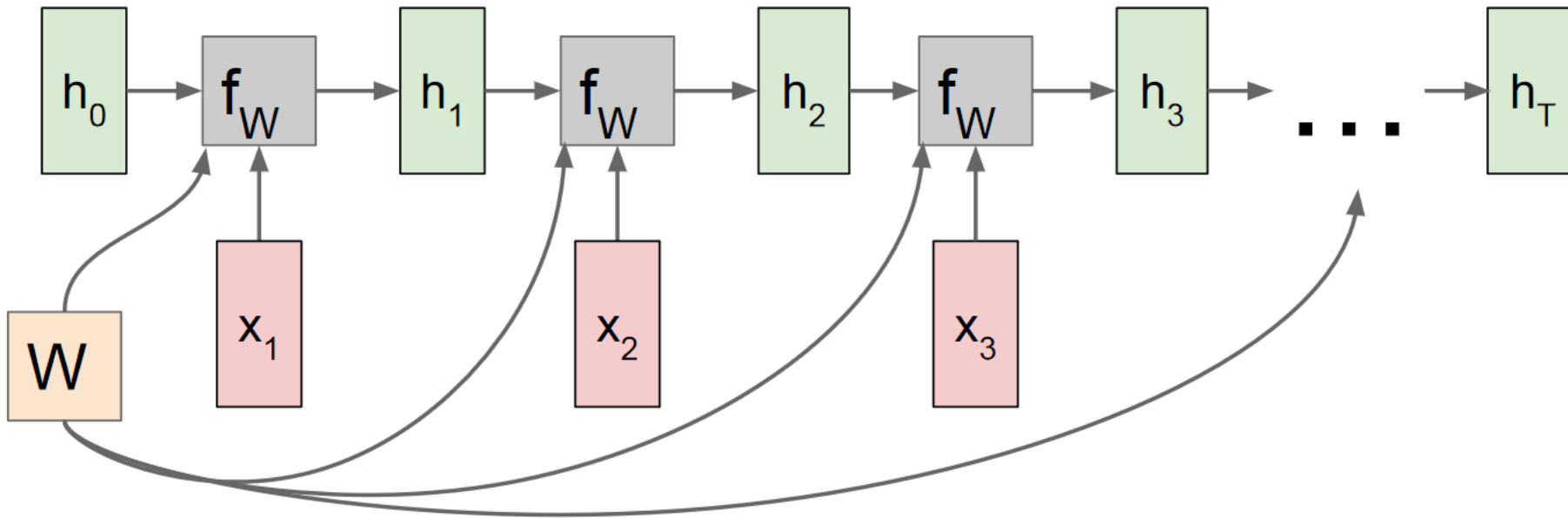
$$h_t = \tanh(\mathbf{W}_{hh}^T h_{t-1} + \mathbf{W}_{xh}^T x_t)$$

Input Vector

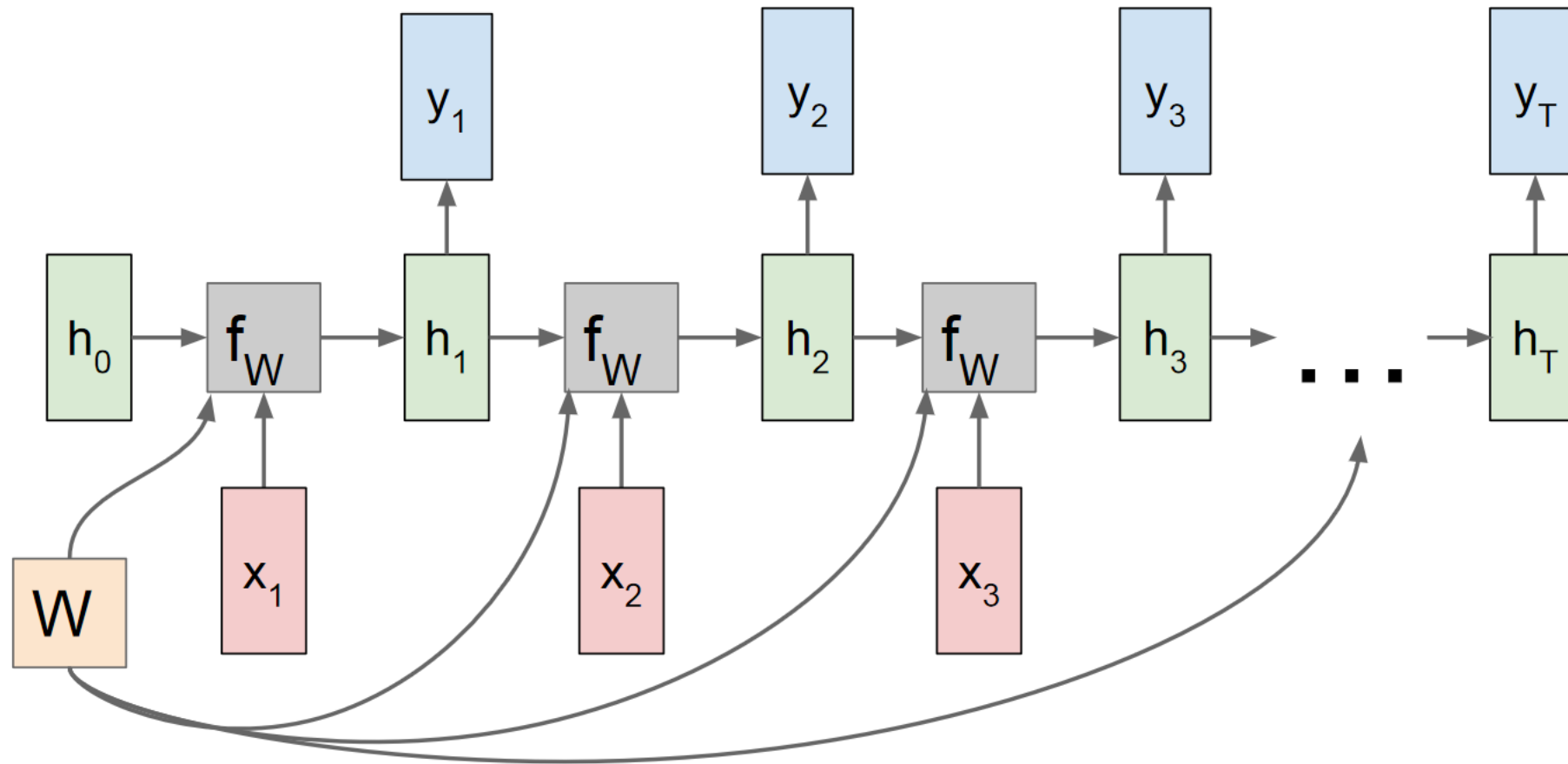
$$x_t$$

RNN: Computational Graph

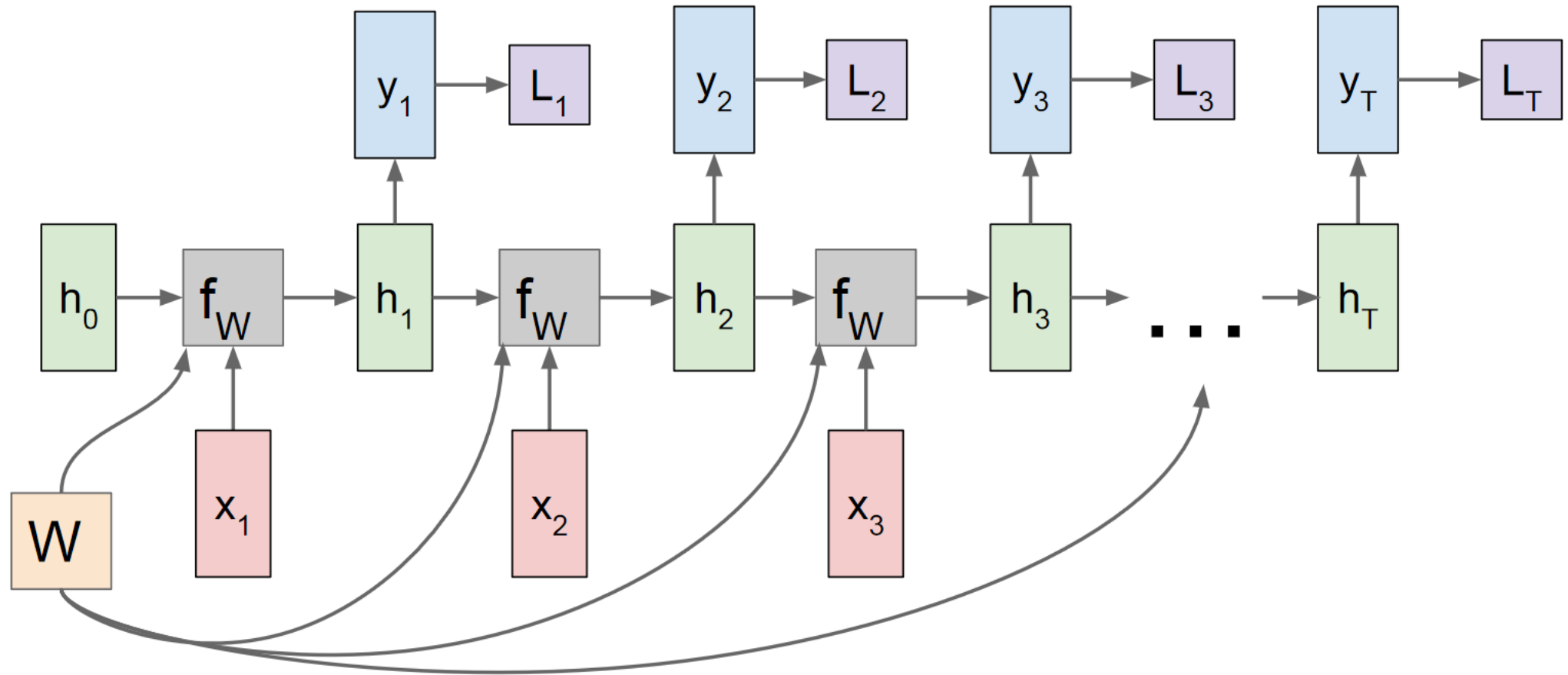
Re-use the same weight matrix at every time-step



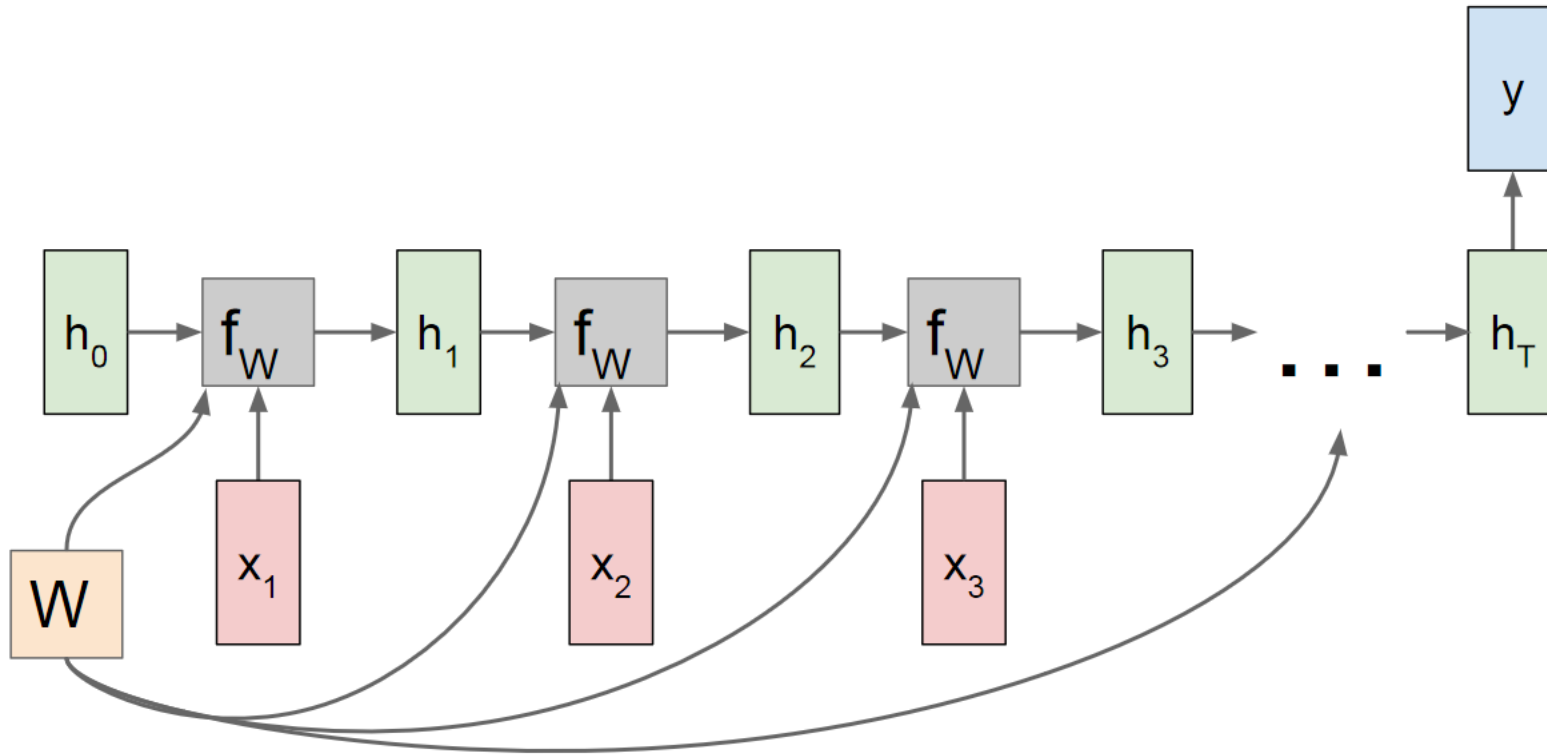
RNN: Computational Graph: Many to Many



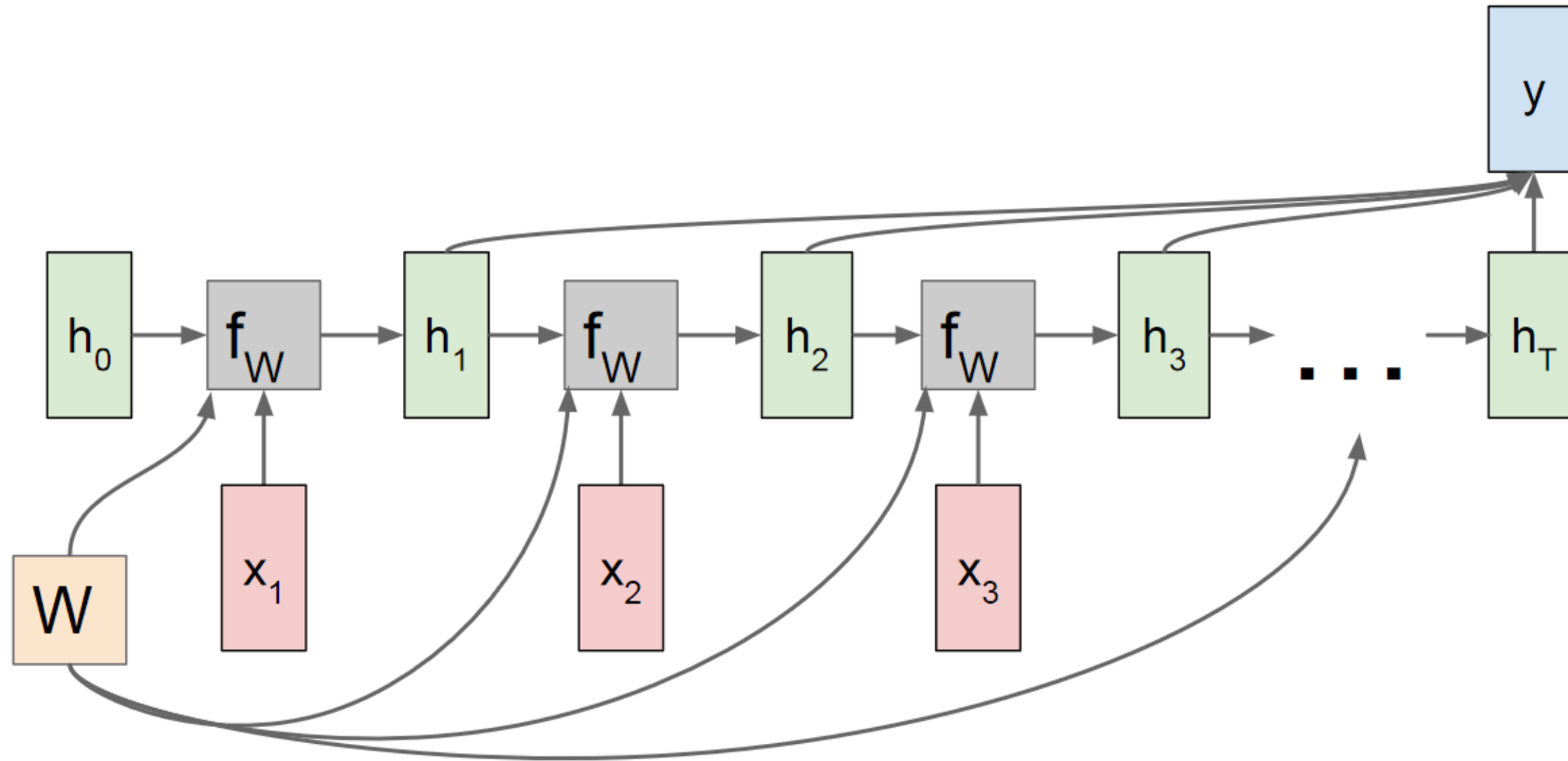
RNN: Computational Graph: Many to Many



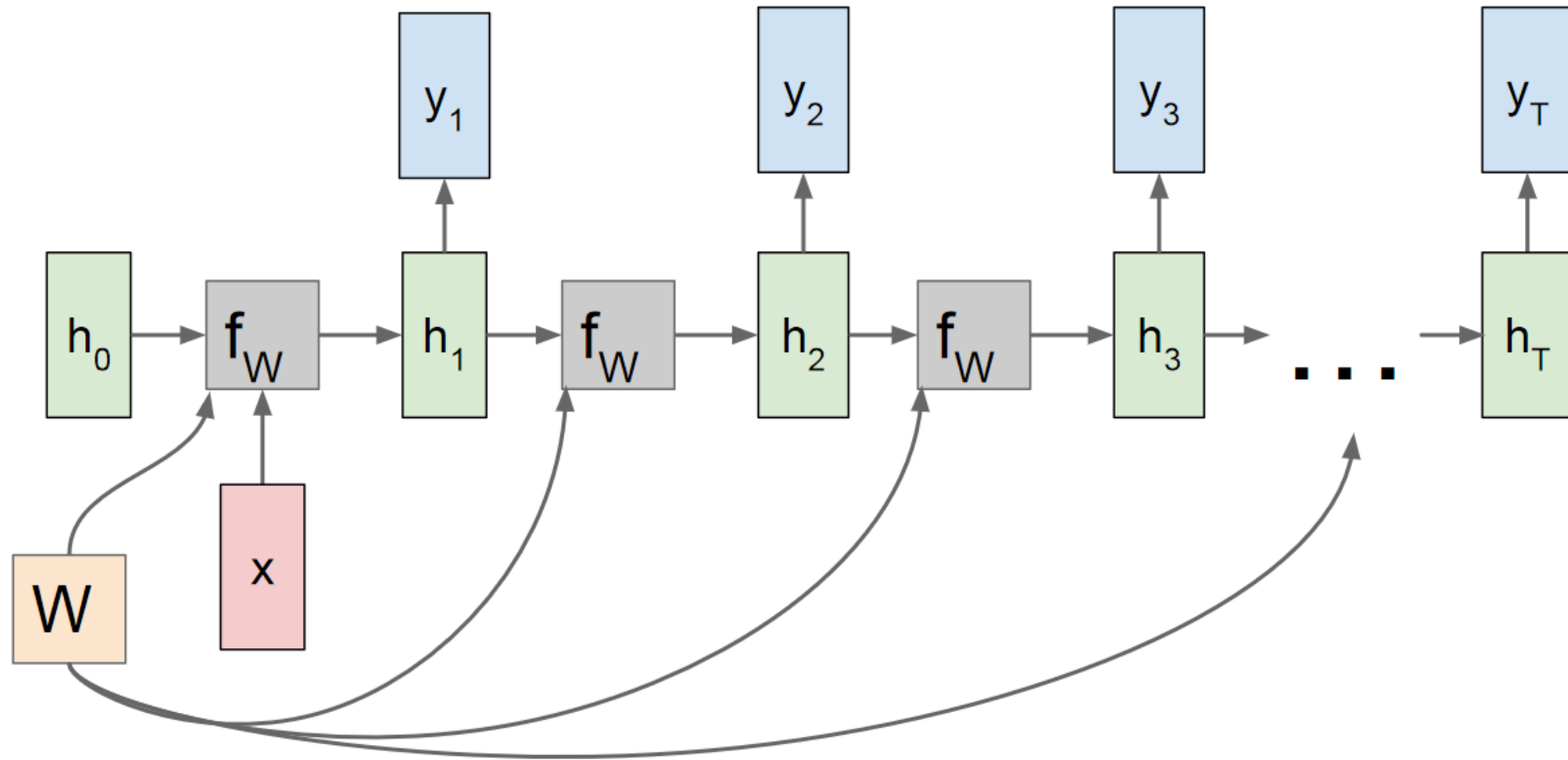
RNN: Computational Graph: Many to One



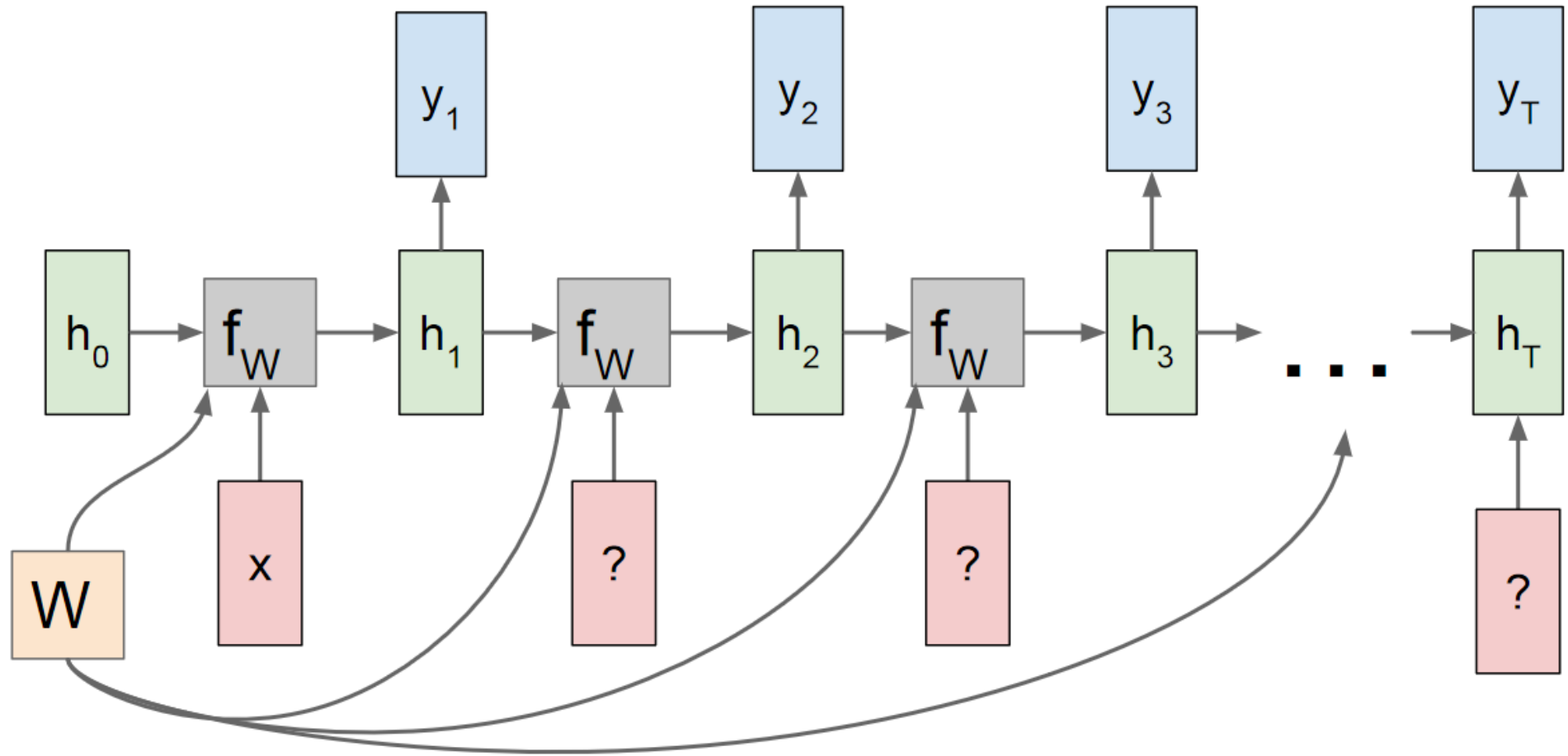
RNN: Computational Graph: Many to One



RNN: Computational Graph: One to Many

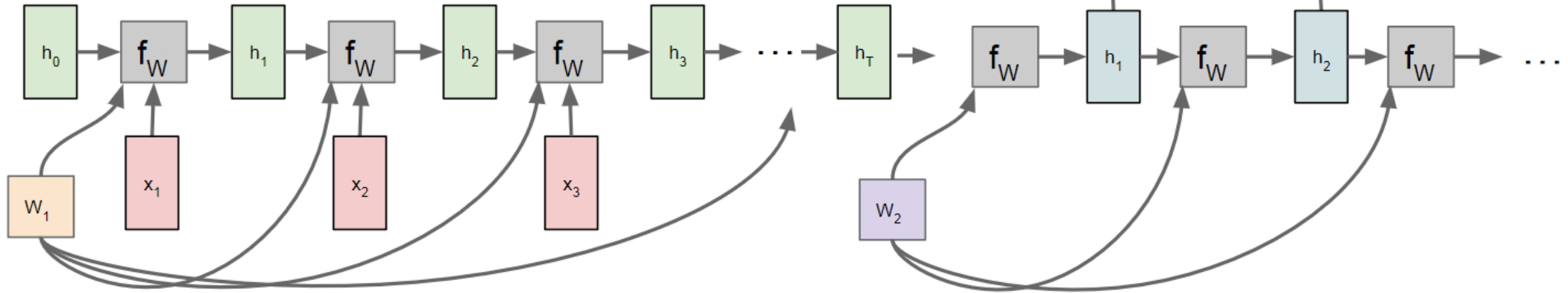


RNN: Computational Graph: One to Many



Sequence to Sequence: Many-to-one + one-to-many

Many to one: Encode input sequence in a single vector



RNN tradeoffs

RNN Advantages:

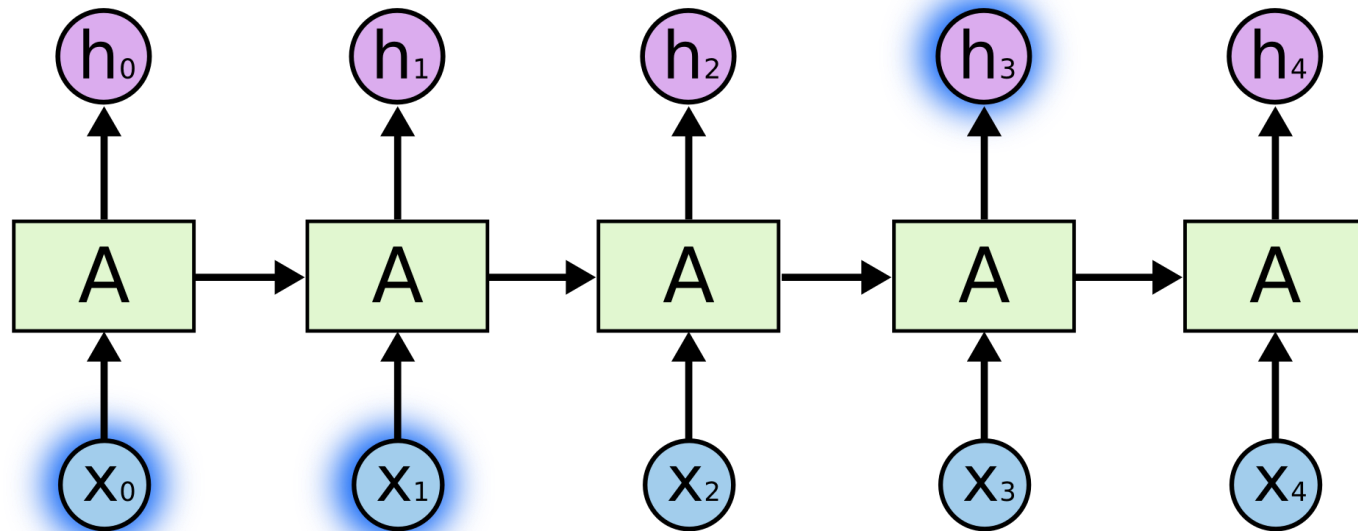
- Can process any length input
- Computation for step t can (in theory) use information from many steps back
- Model size doesn't increase for longer input
- Same weights applied on every timestep, so there is symmetry in how inputs are processed.

RNN Disadvantages:

- Recurrent computation is slow
- In practice, difficult to access information from many steps back

The Problem of Long-Term Dependencies

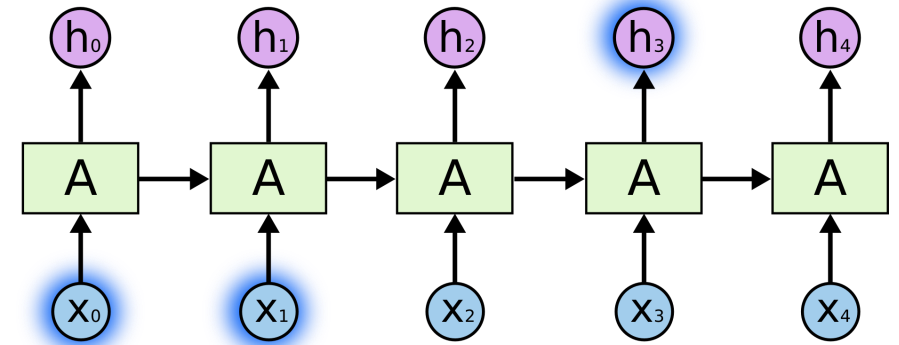
“the clouds are in the *sky*,”



The Problem of Long-Term Dependencies

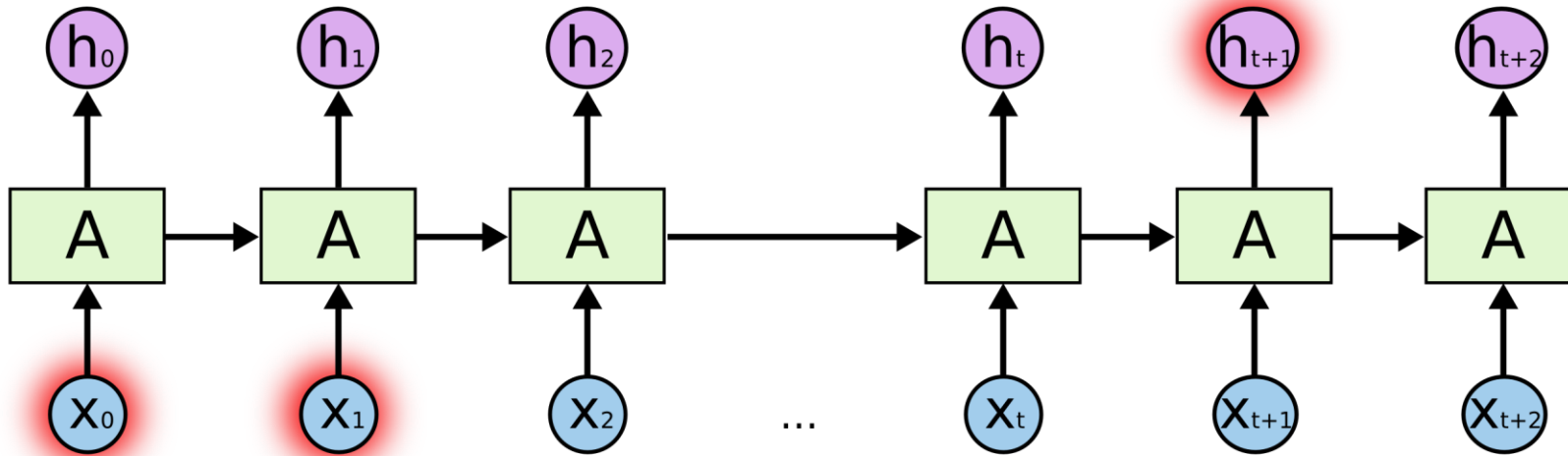
“the clouds are in the *sky*,”

In such cases, where the gap between the relevant information and the place that it's needed is small, RNNs can learn to use the past information.



The Problem of Long-Term Dependencies

“I grew up in France... I speak fluent *French*.”

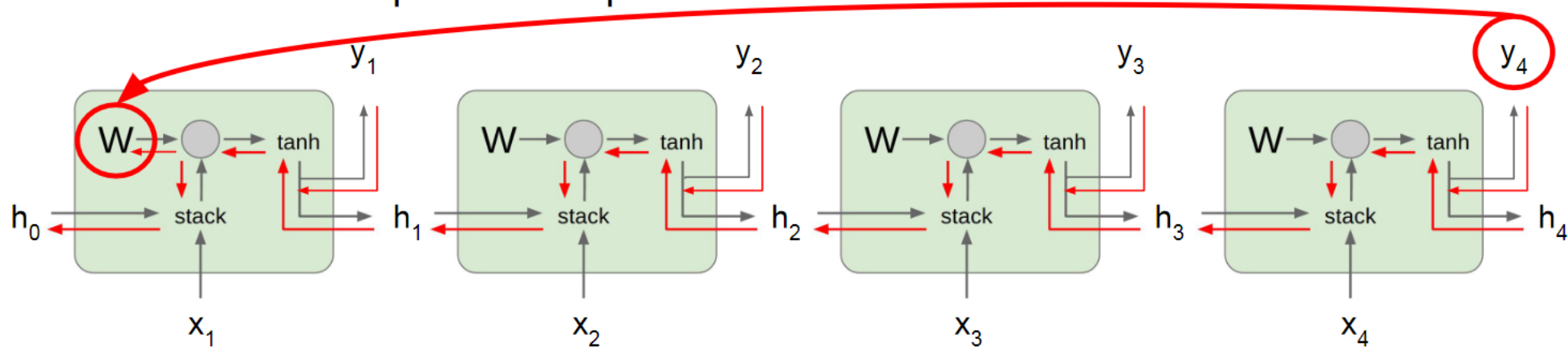


As the gap grows, RNNs become unable to learn to connect the information.

The Problem of Long-Term Dependencies

Gradients over multiple time steps:

ICML 2013



What if we assumed no non-linearity?

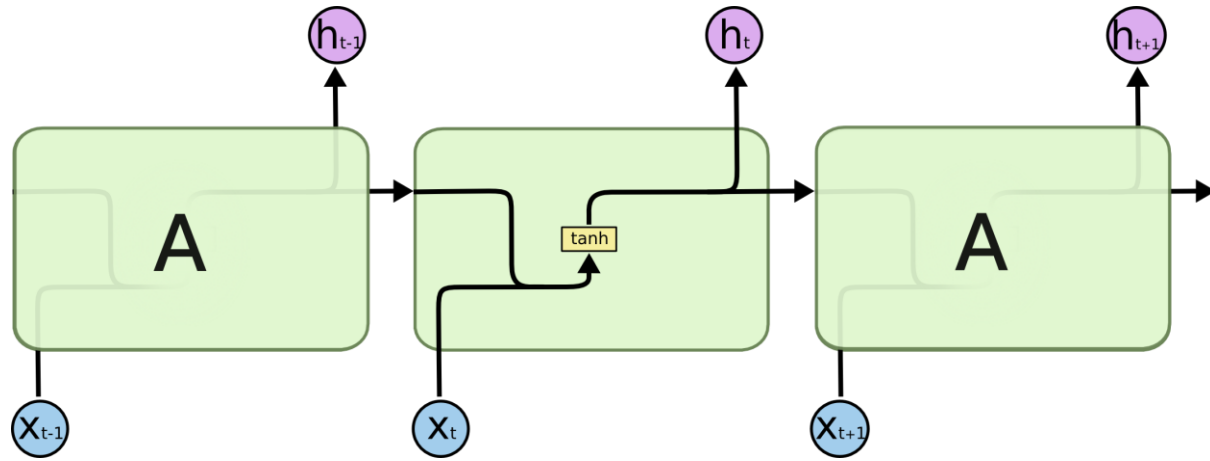
$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$

Largest singular value > 1 :
Exploding gradients

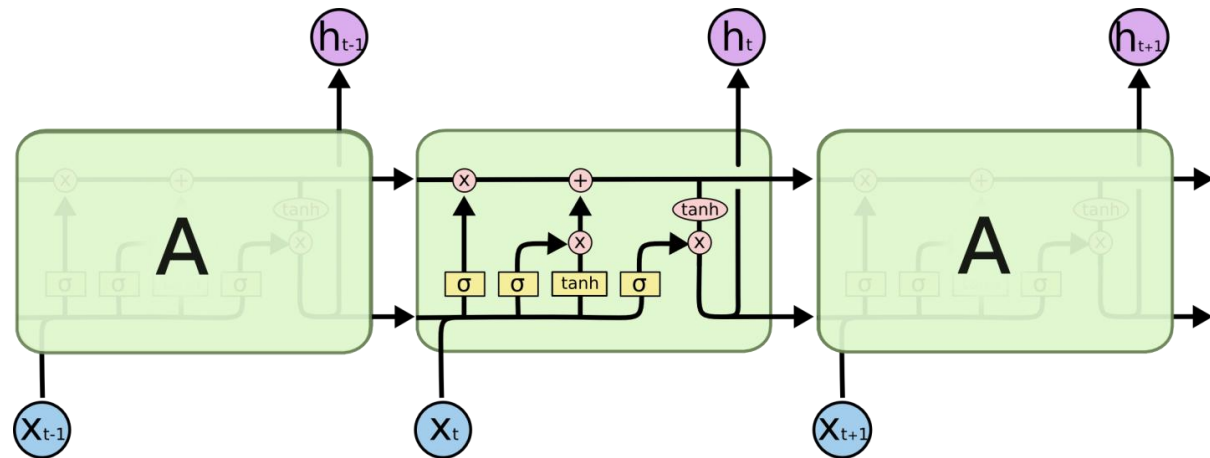
$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \boxed{W_{hh}^{T-1}} \frac{\partial h_1}{\partial W}$$

Largest singular value < 1 :
Vanishing gradients

Long Short Term Memory networks



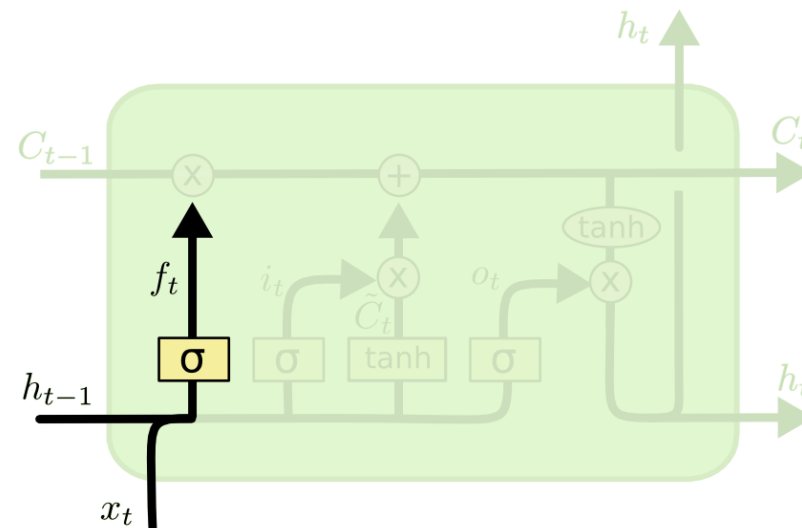
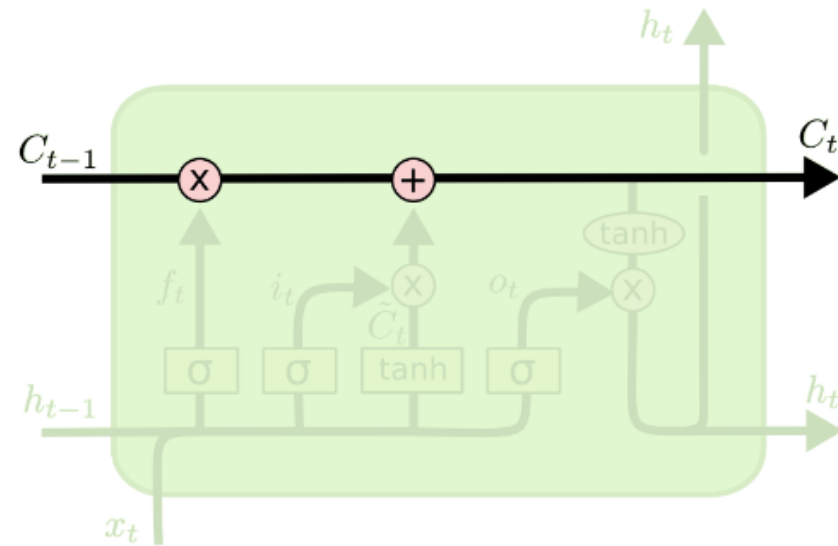
Simple RNN



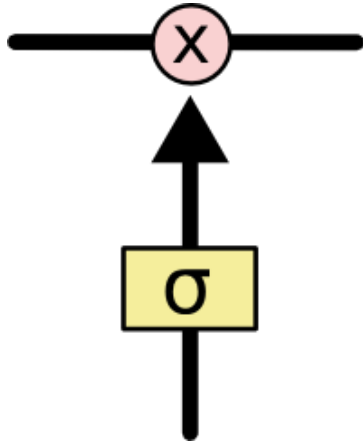
LSTM

Core Component of LSTMs

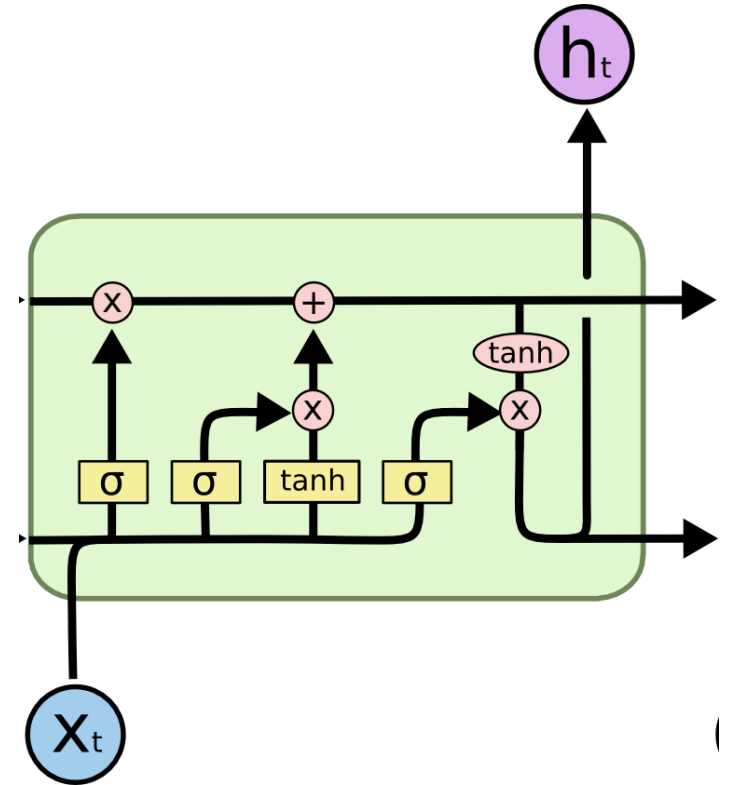
- long-term memory
- short-term memory
- Input (add new info)



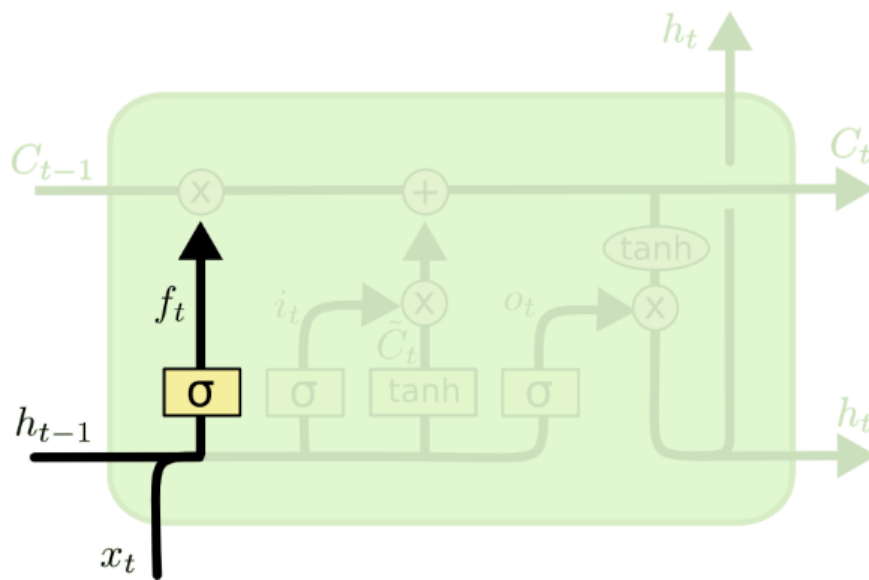
Gate



- Forget gate
- Input gate
- Output gate

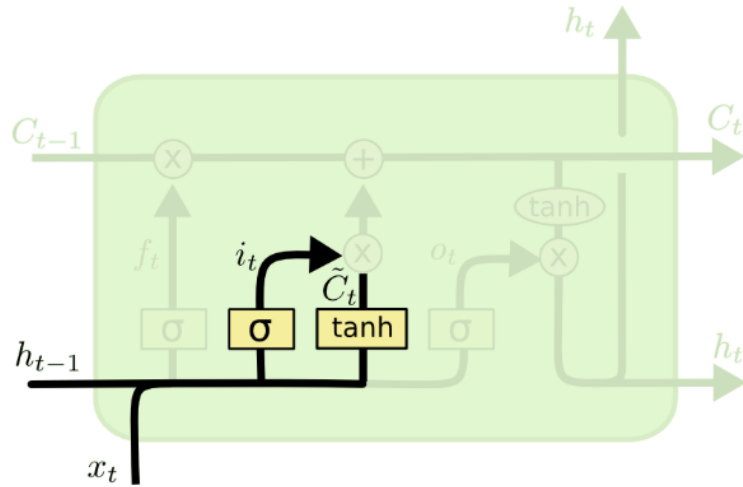


Forget gate



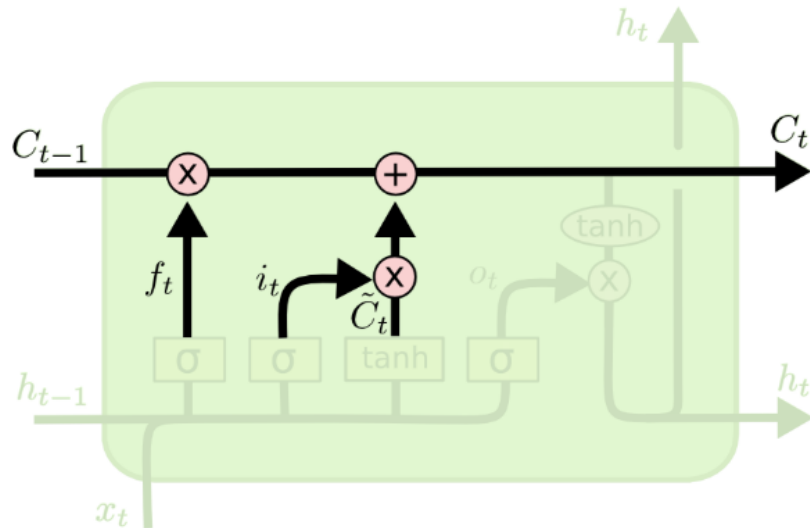
$$f_t = \sigma (W_f \cdot [h_{t-1}, x_t] + b_f)$$

Input gate & Candidate memory



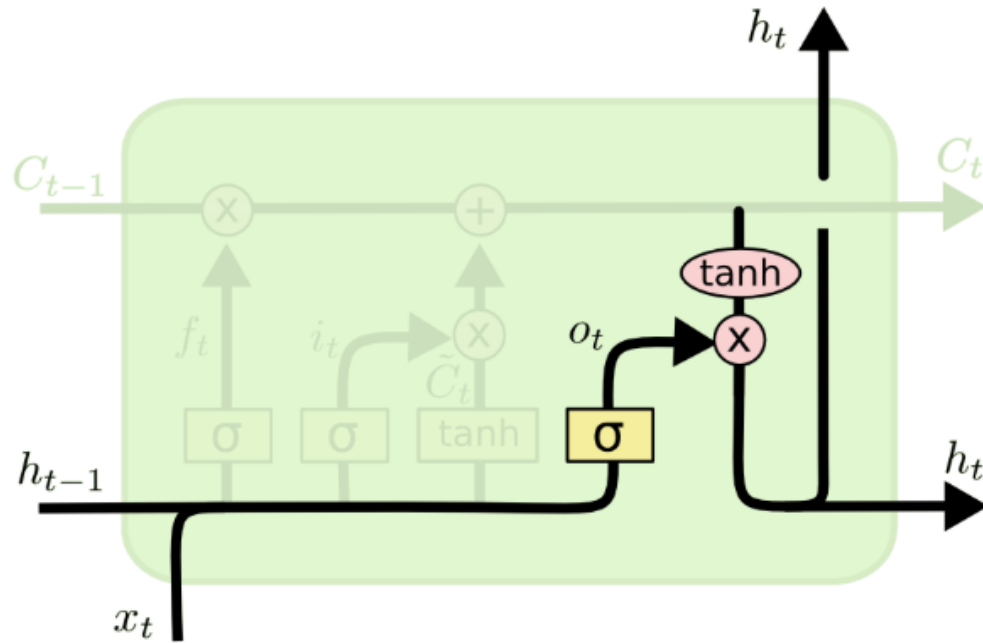
$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

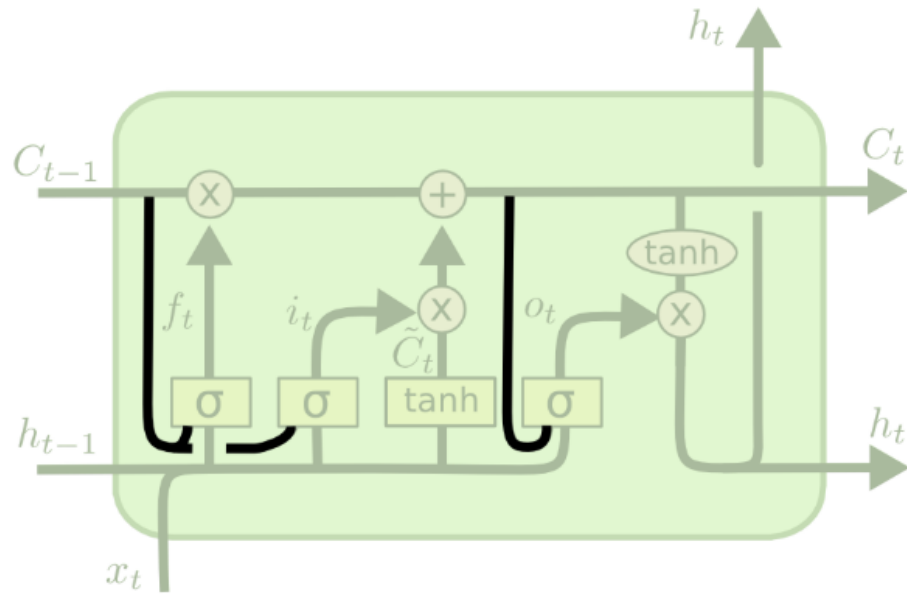
Output gate



$$o_t = \sigma (W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh (C_t)$$

LSTM Variants

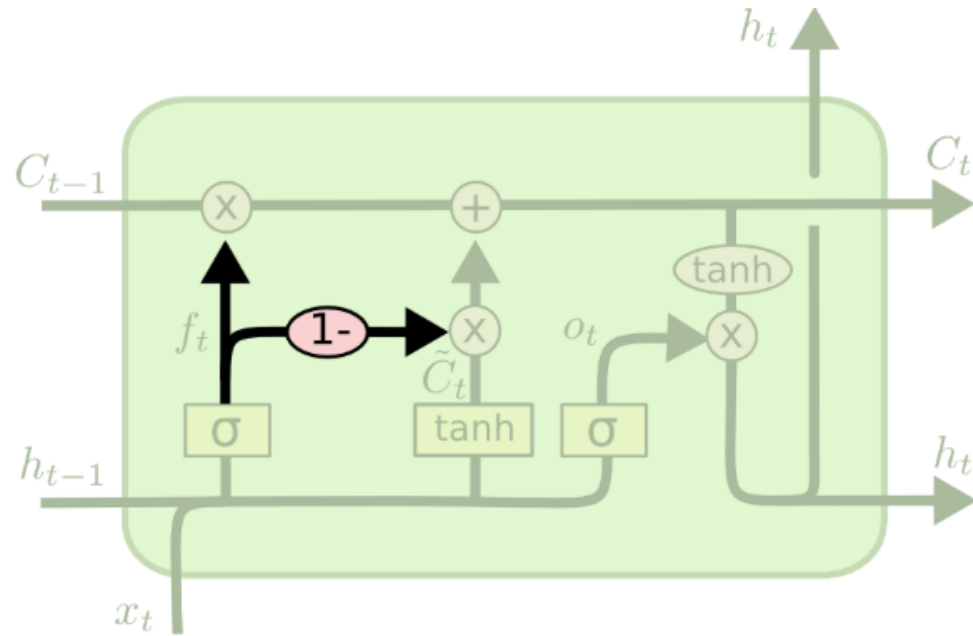


$$f_t = \sigma (W_f \cdot [C_{t-1}, h_{t-1}, x_t] + b_f)$$

$$i_t = \sigma (W_i \cdot [C_{t-1}, h_{t-1}, x_t] + b_i)$$

$$o_t = \sigma (W_o \cdot [C_t, h_{t-1}, x_t] + b_o)$$

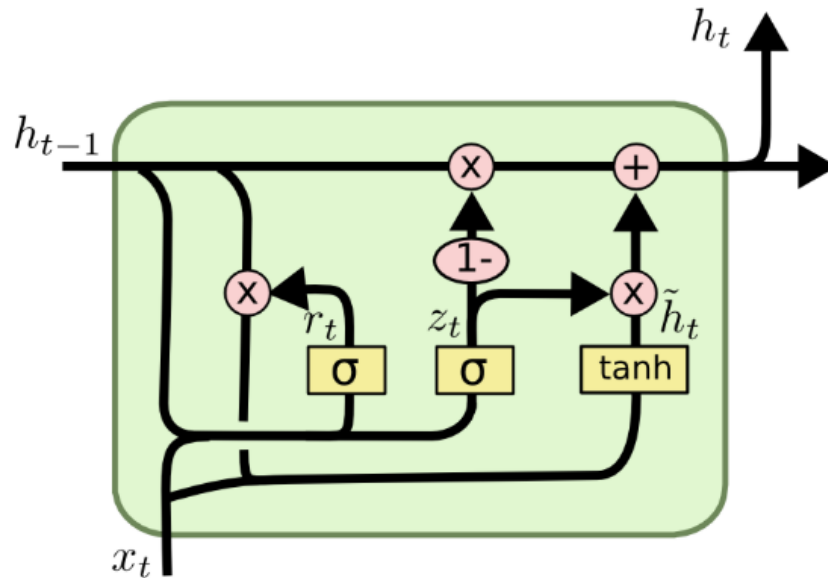
LSTM Variants



$$C_t = f_t * C_{t-1} + (1 - f_t) * \tilde{C}_t$$

LSTM Variants

GRU



$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t])$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$